MACHINE LEARNING AND DEEP LEARNING IN C#

Python became the **default language for AI/ML research** starting around 2010–2015. Every major framework, TensorFlow, PyTorch, Keras was **built from the ground up in Python**. The entire mathematical foundation (NumPy, SciPy) is Python-native.

C# was built for enterprise software, Windows apps, and game development, etc... Microsoft never originally designed C# with scientific computing in mind

1. C# Supports Classical ML (ML.NET)

Microsoft built **ML.NET** specifically to bring classical ML into the C#. It works well because:

Classical ML algorithms are simpler to port

- Algorithms like **Decision Trees, Linear Regression, SVM, K-Means** are mathematically straightforward
- They don't need GPU computation or complex tensor operations
- They run fine on **CPU with normal memory**
- Microsoft could implement these from scratch **natively in C#**

ML.NET Architecture is native C#

- Written fully in C# and .NET
- No dependency on Python runtime
- Tight integration with Visual Studio, Azure, ASP.NET

Data pipeline fits C# strengths

- Classical ML needs structured tabular data, **rows and columns**
- C# is great at handling structured data with strong typing
- `IDataView` pipeline is similar to `scikit-learn` pipelines in Python

Inference (Prediction) is lightweight

- After training a Decision Tree or Regression model, predicting one result is just **a few math operations**
- C# handles this perfectly without needing GPU or special hardware.

DEMO IN ML.NET

- I created a small sample program where I provided sample data for house price prediction in CSV files. This is because ML.NET cannot directly access Excel files, so the data must be in CSV format.
- In the dataset, I included only 5 rows and 3 columns. Using this small dataset, the model is able to make predictions. I implemented this as a demo in Visual Studio using ML.NET.

- For the algorithm, I used a simple one called SDCA, and it predicts correctly for the small dataset. However, when I provide a large amount of data, the model training takes a significant amount of time.

Sample data

```
Size,Bedrooms,Price
600,1,150000
800,2,200000
1000,2,250000
1200,3,300000
1500,3,350000
1800,4,400000
```

Coding part:

```
MLAppDemo                HouseData                Bedrooms

0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Program Started");
        var context = new MLContext();

        // Load data
        var data = context.Data.LoadFromTextFile<HouseData>(
            path: "C:\\Users\\HP\\source\\repos\\MLAppDemo\\excel\\housing.csv",

            hasHeader: true,
            separatorChar: ',');

        // Split data
        var split = context.Data.TrainTestSplit(data, testFraction: 0.2);

        // Define pipeline
        var pipeline = context.Transforms.Concatenate("Features", "Size", "Bedrooms")
                        .Append(context.Regression.Trainers.Sdca(labelColumnName: "Price"));

        // Train model
        var model = pipeline.Fit(split.TrainSet);

        // Test prediction
        var predictor = context.Model.CreatePredictionEngine<HouseData, HousePrediction>(model);

        var sample = new HouseData
        {
            Size = 1300,
            Bedrooms = 3
        };

        var result = predictor.Predict(sample);

        Console.WriteLine($"Predicted Price: {result.Price}");
        Console.ReadLine();
    }
}
```

```
3 references
public class HouseData
{
    [LoadColumn(0)]
    1 reference
    public float Size { get; set; }

    [LoadColumn(1)]
    1 reference
    public float Bedrooms { get; set; }

    [LoadColumn(2)]
    0 references
    public float Price { get; set; }
}

// Output prediction class
1 reference
public class HousePrediction
{
    [ColumnName("Score")]
    1 reference
    public float Price { get; set; }
}
```

OUTPUT

```
C:\Users\HP\source\repos\MLAppDemo\bin\Debug\net10.0\MLAppDemo.exe
Program Started
Predicted Price: 212654.22
```

2. Deep Learning Fails in C#

Reason 1: TensorFlow is NOT Native to C#

- TensorFlow is written in **C++ at the core**, but its **primary API and design is built around Python**
- All official documentation, examples, and support is Python-first
- Google designed TensorFlow assuming Python as the entry point

What TensorFlow.NET actually does:

- It is a **third-party wrapper** (not made by Google)
- It calls the underlying C++ TensorFlow library through **P/Invoke bindings**
- It translates C# code → C++ TensorFlow calls → result back to C#
- This translation layer causes:
    - **Version lag** — TensorFlow.NET is always 1–2 versions behind Python TF
    - **Missing features** — Not all TF functions are wrapped
    - **Bugs** — The binding layer itself introduces errors
    - **Poor error messages** — Errors from C++ layer are hard to debug in C#

And also tensorflow.net is missing some features model,

Missing TensorFlow features

- keras high-level(not full suport, partial)support only)
- Custom training loops
- GPU training
- TF.data pipeline
- TF.distribute
- TF Lite
- TensorBoard Integration
- Eager execute(partial support)
- Model saving/loading(only basic save model)
- Pre-trained model hub

Reason 2: No Native Tensor Operations

- Deep Learning runs entirely on **tensors**, multi-dimensional arrays of numbers
- Operations like matrix multiplication, backpropagation, convolution happen on tensors at massive scale

Python has NumPy as foundation:

- NumPy provides **native tensor/array operations** in Python
- TensorFlow and PyTorch are **built on top of NumPy concepts**
- Python can manipulate tensors directly and intuitively

C# has NO NumPy equivalent:

- There is no mature, native tensor library in C#
- `MathNet.Numerics` exists but it is for basic linear algebra — not deep learning scale
- Without native tensor ops, you cannot build DL from scratch in C#
- Every tensor operation has to be **routed through wrappers** adding overhead and instability

Reason 3: GPU Support is Broken

GPU matters for Deep Learning:

- Training a neural network means doing **billions of math operations** (matrix multiplications)
- CPUs have 8–32 cores — too slow for this
- GPUs have **thousands of cores** running in parallel — made for this
- Without GPU, training a ResNet or BERT model takes **days on CPU vs hours on GPU**

Python GPU Support:

- TensorFlow and PyTorch directly communicate with **NVIDIA CUDA** (GPU programming framework)
- Install TF + CUDA drivers → GPU training works immediately
- Full support for multi-GPU, mixed precision, memory optimization

C# GPU Support:

- TensorFlow.NET GPU setup is **extremely complex and fragile**
- Requires matching exact versions of: TensorFlow.NET + CUDA + cuDNN + Windows/Linux

Frequent errors like `CUDA not found`, `DLL missing`, `version mismatch`

- ML.NET has **zero native GPU training support**
- TorchSharp (PyTorch for C#) has partial GPU support but is unstable

Real impact:

- HEre can't realistically train a CNN, RNN, or Transformer in C# with GPU
- You are stuck with CPU-only, which makes DL training **practically impossible** at scale

Reason 4: Backpropagation & Autograd Not Properly Implemented

- It is the algorithm that **trains neural networks**
- It calculates how much each weight contributed to the error and adjusts them
- This requires **automatic differentiation (autograd)**, automatically computing gradients of any function

Python:

- PyTorch has `autograd` the most powerful automatic differentiation engine
- TensorFlow has `GradientTape` — tracks operations and computes gradients automatically
- These are **deeply integrated** into the framework core

C#:

- TorchSharp has partial autograd but **custom gradient operations are unsupported**
- TensorFlow.NET's `GradientTape` is incomplete — breaks on complex architectures
- You **cannot build custom loss functions or custom layers** reliably
- This means architectures like **Transformers, GANs, custom CNNs** are nearly impossible to train in C#

Reason 5: Pre-trained Models

Python ecosystem:

- HuggingFace has **200,000+ pre-trained models** (BERT, GPT, ViT, Whisper, etc.)
- TensorFlow Hub, PyTorch Hub — download and use any model in 3 lines of code
- Fine-tuning a BERT model for your task = 10 lines of Python

C# ecosystem:

- HuggingFace has **no C# SDK**
- TF Hub models cannot be directly loaded in TensorFlow.NET
- You cannot fine-tune BERT, GPT, or any LLM in C#
- Only option: export model to **ONNX format in Python**, then load in C# for inference only
- You get inference, but **no training, no fine-tuning, no customization**